

Yesterday 3:31 PM

TCP vs UDP.

HTTP vs HTTPS.

What happens when you type:

<https://google.com>

DNS resolution.

TCP handshake.

HTTP request/response.

Keep-alive connections.

HTTP/1.1 vs HTTP/2.

What is connection timeout?

Show more ▾



Absolutely. For a **5-year Java/Spring Boot backend interview**, these networking/HTTP questions are very important because interviewers often expect you to explain not just definitions, but **what actually happens in a production system**.

1. TCP vs UDP

 Loading

TCP

UDP

Connection-oriented

Connectionless

TCP	UDP
Reliable delivery	No delivery guarantee
Ordered packets	No ordering guarantee
Retransmits lost packets	Doesn't retransmit
Flow/congestion control	No built-in reliability
Higher overhead	Lower overhead
HTTP/HTTPS commonly uses TCP	DNS, streaming, gaming can use UDP

TCP example

Suppose you send:

Packet 1
Packet 2
Packet 3



If packet 2 is lost, TCP can detect the loss and retransmit it.

UDP

UDP essentially says:

"Here's the packet. I'm not going to establish a reliable connection or guarantee delivery."

Interview answer:

TCP is preferred when reliable, ordered delivery is required, while UDP is preferred when low latency is more important than guaranteed delivery.

2. HTTP vs HTTPS

HTTP

HTTP = HyperText Transfer Protocol.

Example:

`http://example.com`



Data is generally sent without TLS encryption.

HTTPS

HTTPS = HTTP + TLS.

`https://example.com`



HTTPS provides:

1. **Encryption** — others can't easily read the traffic.
2. **Integrity** — data shouldn't be silently modified in transit.
3. **Authentication** — TLS certificates help verify the server's identity.

Typical flow:

```
Client
  |
  | TLS handshake
  ↓
Server
  |
  | encrypted HTTP communication
  ↓
Client
```



Interview point: HTTPS doesn't mean a completely different application protocol. It's HTTP carried over a TLS-secured connection.

3. What happens when you type `https://google.com`?

This is a **very common 5-year interview question**.

Think of it as:

```
Browser
  ↓
DNS
  ↓
IP address
```



↓
TCP connection
↓
TLS handshake
↓
HTTP request
↓
Server
↓
HTTP response
↓
Browser renders page

Let's break it down.

Step 1 — Browser parses the URL

You enter:

https://google.com



The browser identifies:

Protocol = HTTPS
Host = google.com
Port = 443



HTTPS normally uses port **443**.

4. DNS Resolution

The browser needs to know:

"Which IP address corresponds to google.com?"

It performs DNS resolution.

Simplified:

Browser
↓
OS DNS cache



```

↓
DNS resolver
↓
Root DNS
↓
.com DNS
↓
Authoritative DNS
↓
IP address

```

For example, conceptually:

google.com → some IP address



The actual address can vary because large services commonly use distributed infrastructure.

Why DNS?

Humans remember:

google.com



Machines communicate using IP addresses.

5. TCP Handshake

Once the client has an IP address, it establishes a TCP connection.

The famous **3-way handshake**:

```

Client                                Server
SYN ----->
<----- SYN + ACK
ACK ----->

```



Step 1

Client:

SYN



Meaning:

"I want to establish a TCP connection."

Step 2

Server:

SYN + ACK



Meaning:

"I received your request and I'm willing to establish the connection."

Step 3

Client:

ACK



Connection established.

6. TLS Handshake

Because this is HTTPS, TCP isn't enough.

The client and server now establish a secure TLS session.

Simplified:

TCP connection



TLS handshake



Secure encrypted connection



During TLS negotiation, the client and server establish cryptographic parameters and the server proves its identity using its certificate.

Then HTTP data can be encrypted.

7. HTTP Request/Response

Now the browser can send an HTTP request.

Conceptually:

```
</> http
```

```
GET / HTTP/1.1
```

```
Host: google.com
```



Server processes it and sends a response:

```
HTTP status
```

```
Headers
```

```
Body
```



For example:

```
200 OK
```



The browser then processes the returned resources and renders the page.

8. Keep-Alive Connections

Imagine a webpage requires:

```
HTML
```

```
CSS
```

```
JavaScript
```

```
Images
```

```
API calls
```



Without connection reuse, the client might repeatedly establish connections.

That creates overhead.

With persistent connections:

```
TCP connection
```

```
|
```



```
+---- Request 1
+---- Response 1
+---- Request 2
+---- Response 2
+---- Request 3
+---- Response 3
```

The connection can be reused for multiple requests.

Why is this useful?

It reduces:

- TCP handshake overhead
- TLS handshake overhead
- latency
- connection establishment cost

HTTP/1.1 uses persistent connections by default unless configured otherwise.

9. HTTP/1.1 vs HTTP/2

This is particularly important for backend interviews.

HTTP/1.1	HTTP/2
Text-based protocol	Binary framing
Requests traditionally compete for connection resources	Multiplexing
Multiple connections often used	Multiple streams over one connection
Head-of-line issues at HTTP level	Better request multiplexing
No header compression like HPACK	HPACK header compression
Less efficient for many concurrent requests	More efficient

HTTP/1.1

Conceptually:

Connection



Request → Response

Request → Response

Request → Response



HTTP/1.1 can pipeline in theory, but practical deployment has historically relied heavily on multiple connections rather than HTTP/1.1 pipelining.

HTTP/2

One connection can contain multiple streams:

Single TCP connection



Stream 1 → Request A

Stream 2 → Request B

Stream 3 → Request C

Stream 4 → Request D

This is called **multiplexing**.

Important interview point

Don't say:

"HTTP/2 removes TCP head-of-line blocking."

That's not completely correct.

HTTP/2 multiplexes HTTP streams, but because HTTP/2 commonly runs over TCP, **TCP-level packet loss can still cause head-of-line blocking**.

HTTP/3 addresses this by using QUIC over UDP.

10. What is Connection Timeout?

Connection timeout means:

How long the client waits while trying to establish a connection before giving up.

Example:

```
</> Java
```

```
connectTimeout = 3 seconds
```



Suppose your application calls:

```
Order Service
```



but the server cannot be reached.

The client tries to establish the connection.

After 3 seconds:

```
Connection timeout
```



The request hasn't even reached the application successfully.

11. Read Timeout vs Connection Timeout

Very important distinction.

Connection timeout

Problem happens while **establishing the connection**.

```
Client
```

```
|  
|---- connect ----> Server  
|  
| ✗ cannot connect
```



Example:

```
connectTimeout = 3 sec
```



Read timeout

Connection was successfully established, but the client is waiting for the server's response/data.

Client

```
|  
|---- request ---->  
|  
|    Server processing...  
|  
|    Server processing...  
|  
|    Server processing...  
|  
✖ read timeout
```



Example:

```
readTimeout = 5 sec
```



Interview answer

Connection timeout controls how long we wait to establish the connection, whereas read timeout controls how long we wait for data after the connection has been established.

12. What is Idempotency?

An operation is **idempotent** if performing it multiple times has the same intended final effect as performing it once.

For example:

```
</> http  
  
PUT /users/123
```



with:

```
</> JSON  
  
{  
  "name": "Manoj"  
}
```



Calling it once:

```
name = Manoj
```



Calling it 10 times:

```
name = Manoj
```



The intended final state is the same.

HTTP methods

Generally:

Method	Idempotent?
GET	Yes
PUT	Yes
DELETE	Yes
POST	Generally no
PATCH	Depends on implementation

Important: Idempotency is about the **intended effect**, not necessarily identical responses, logs, timestamps, etc.

13. GET vs POST vs PUT vs PATCH

GET

Used to retrieve data.

```
</> http
```



```
GET /users/123
```

Generally:

- Safe
- Idempotent
- Should not modify server state

POST

Usually used to create a resource or trigger an operation.

```
</> http
```



```
POST /users
```

Example:

```
</> JSON
```



```
{  
  "name": "Manoj"  
}
```

Calling it twice could create:

```
User 101
```



```
User 102
```

Therefore POST is generally **not idempotent**.

PUT

Usually used to create/replace a resource at a known URI.

```
</> http
```



```
PUT /users/123
```

Conceptually:

```
Replace/update user 123 with this representation
```



Typically idempotent.

PATCH

Used for partial modification.

```
</> http
```



```
PATCH /users/123
```

Example:

```
</> JSON
```



```
{  
  "email": "new@email.com"  
}
```

Only the specified part is changed.

PATCH can be idempotent or non-idempotent depending on the operation.

14. What does HTTP 429 mean?

```
429 Too Many Requests
```



It generally means:

The client has sent too many requests in a given period.

Common reason:

```
Rate limiting
```



Example:

```
API limit = 100 requests/minute
```



```
Client sends:  
120 requests/minute
```

```
→ 429
```

A server may also provide:

```
</> http
```



Retry-After

telling the client when it may retry.

Production example

You might have:

```
API Gateway
  ↓
Rate Limiter
  ↓
Microservice
```



The rate limiter protects the service from excessive traffic.

15. What does HTTP 502 mean?

502 Bad Gateway



This generally means:

A gateway/proxy received an invalid or unexpected response from an upstream server.

Example:

```
Client
  ↓
ALB / Gateway / Proxy
  ↓
Service A
  ↓
Service B
```



Suppose Service A expects a valid response from Service B, but Service B returns something invalid or the intermediary cannot obtain a valid upstream response.

You can see:

502 Bad Gateway



Simple mental model

Gateway



bad response from upstream



16. 503 vs 504

This is a **very common microservices interview question**.

503 — Service Unavailable

503 Service Unavailable



Means the service is currently unable to handle the request.

Possible causes:

- Service is down
- No healthy instances
- Service overloaded
- Maintenance
- Kubernetes pods unavailable
- Load balancer has no healthy targets

Think:

"The service isn't available right now."

17. 504 — Gateway Timeout

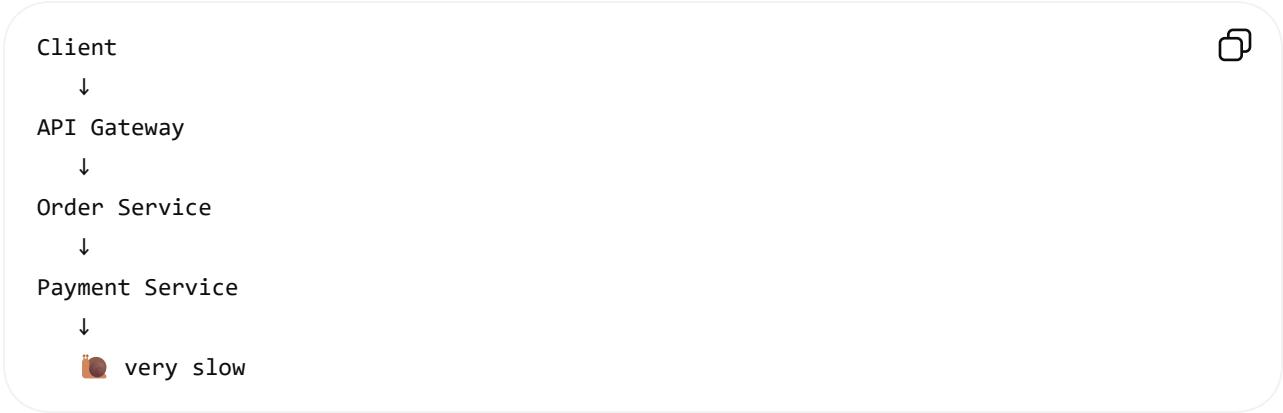
504 Gateway Timeout



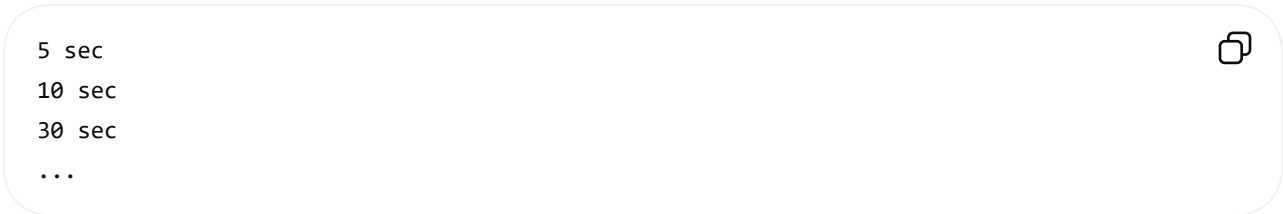
Means:

A gateway/proxy didn't receive a response from the upstream service within the allowed time.

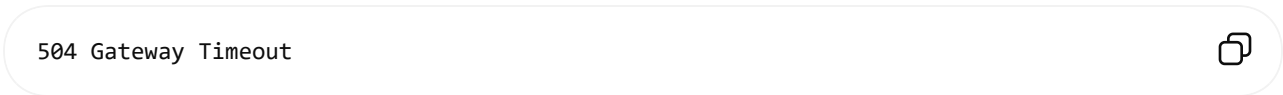
Example:



Gateway waits:



Eventually:



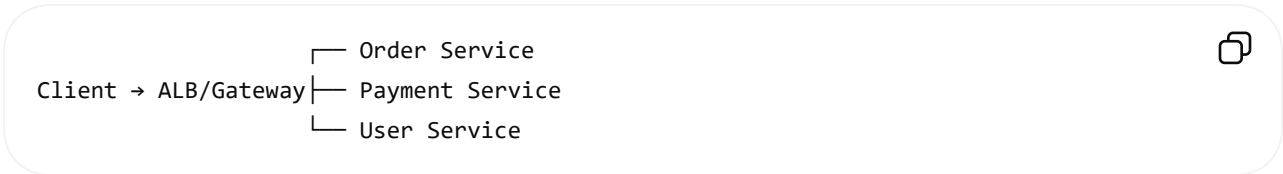
Think:

"The service may be there, but it didn't respond in time."

18. 502 vs 503 vs 504 — Remember This

Status	Meaning	Easy way to remember
502	Bad upstream response	"Upstream responded badly"
503	Service unavailable	"Service isn't available"
504	Upstream timeout	"Service didn't respond in time"

Real microservice example



502

Gateway communicates with upstream but gets an invalid/bad response.

```
Gateway → Service
      ↓
    bad response
      ↓
      502
```



503

No healthy service is available.

```
Gateway → Service
      ↓
    no healthy instance
      ↓
      503
```



504

Service exists but takes too long.

```
Gateway → Service
      ↓
    waiting...
    waiting...
      ↓
    timeout
      ↓
      504
```



★ 5-Year Interview-Level Summary

If the interviewer asks:

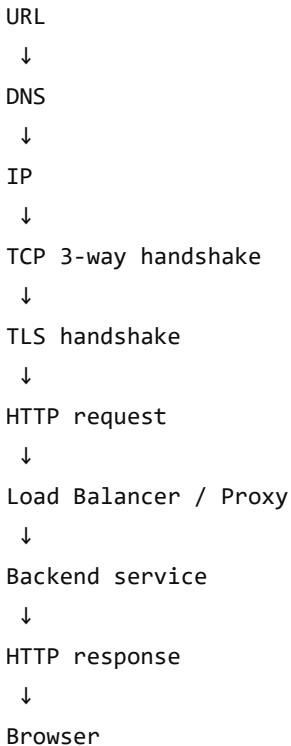
"Explain what happens when I type https://google.com ↗."

A strong answer would be:

"The browser first parses the URL and identifies HTTPS and the host. It resolves the hostname through DNS to obtain an IP address. It then establishes a TCP connection using the three-way handshake. Since it's HTTPS, a TLS handshake establishes a secure

encrypted session. The browser then sends an HTTP request, typically over a persistent connection. The server or intermediary processes the request and returns an HTTP response, which the browser uses to render the page. For subsequent resources, connection reuse can avoid repeatedly establishing TCP and TLS connections."

And if they go deeper:



```
graph TD; URL --> DNS; DNS --> IP; IP --> TCP[TCP 3-way handshake]; TCP --> TLS[TLS handshake]; TLS --> HTTP[HTTP request]; HTTP --> LB[Load Balancer / Proxy]; LB --> Backend[Backend service]; Backend --> HTTP_resp[HTTP response]; HTTP_resp --> Browser;
```

The key thing at 5 years is not just knowing definitions. Be able to connect these concepts to timeouts, load balancers, API gateways, microservices, retries, rate limiting, and production failures.



InputStream vs Reader.
OutputStream vs Writer.
BufferedInputStream.
BufferedReader.
File vs Path.
java.io vs java.nio.
What is NIO?
What is a channel?
What is a buffer?

Blocking vs non-blocking I/O.

How would you process a 10 GB file without loading it

Show more ▾

These are important **Java I/O** questions for a 5-year Java/Spring Boot interview. The main thing is to understand the difference between **bytes**, **characters**, **files**, **channels**, **buffers**, and **streaming**.

1. InputStream vs Reader

The simplest distinction:

InputStream works with bytes; **Reader** works with characters.

InputStream

Used for binary data.

Examples:

- Images
- PDFs
- ZIP files
- Audio
- Raw network data

</> Java



```
InputStream input = new FileInputStream("image.jpg");

int data;
while ((data = input.read()) != -1) {
    // process byte
}
```

Reader

Used for character/text data.

</> Java



```
Reader reader = new FileReader("data.txt");
```

```
int data;
while ((data = reader.read()) != -1) {
    // process character
}
```

Interview comparison

InputStream	Reader
Byte-oriented	Character-oriented
8-bit bytes	Characters
Binary data	Text data
read() returns byte value	read() returns character value

Important

Don't use `InputStream` simply because "everything is bytes."

If you're processing text, using a `Reader` allows Java to handle character encoding appropriately.

2. OutputStream vs Writer

Same concept in the opposite direction.

OutputStream writes bytes; **Writer** writes characters.

OutputStream

</> Java



```
OutputStream out =
    new FileOutputStream("data.bin");

out.write(65);
```

Good for binary data.

Writer

</> Java

```
Writer writer =
    new FileWriter("data.txt");

writer.write("Hello Java");
```

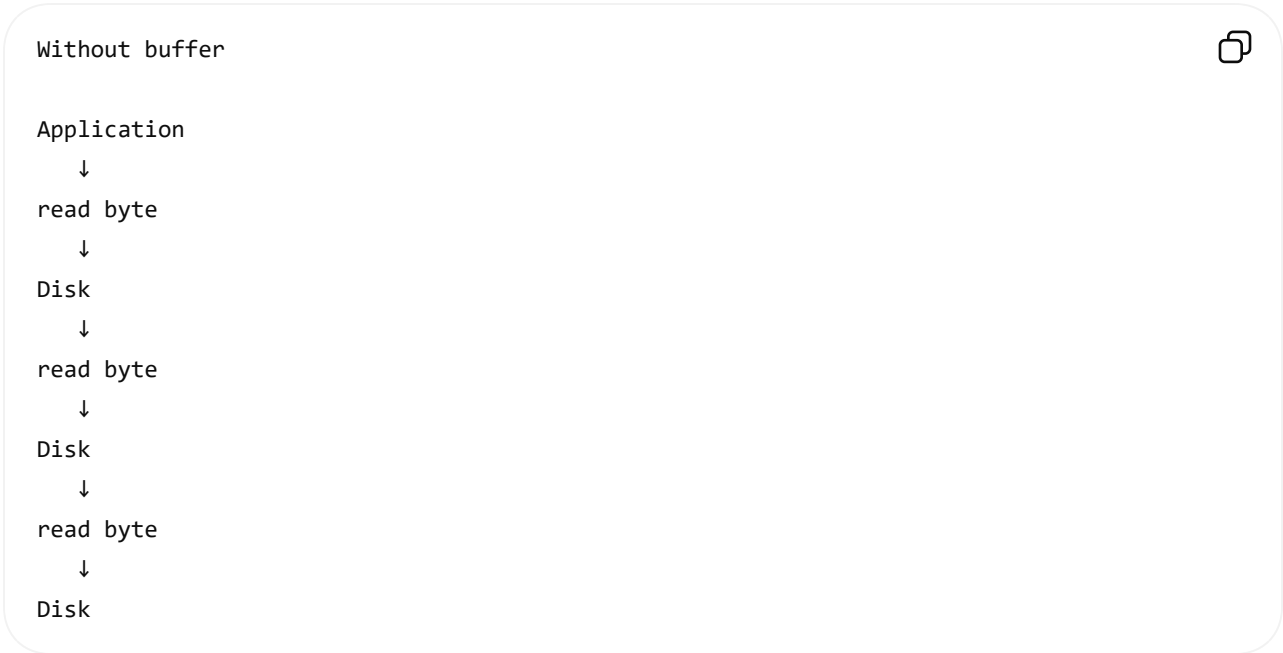
Good for text.

OutputStream	Writer
Bytes	Characters
Binary output	Text output
write(byte[])	write(char[]) / write(String)

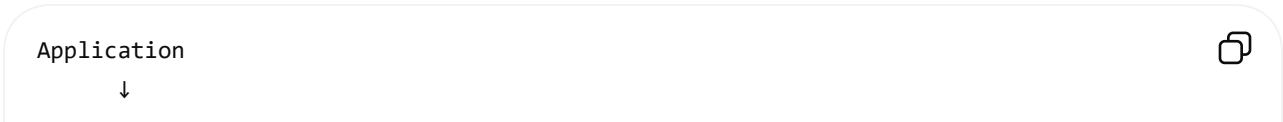
3. BufferedInputStream

BufferedInputStream adds a buffer around an InputStream .

Instead of repeatedly going to the underlying source for every small read:



With buffering:



Buffer



Disk

The buffer reads a larger chunk and then serves smaller reads from memory.

</> Java



```
try (InputStream input =
    new BufferedInputStream(
        new FileInputStream("data.bin"))) {

    int data;

    while ((data = input.read()) != -1) {
        // process
    }
}
```

Why?

Primarily to reduce expensive underlying I/O operations.

4. BufferedReader

`BufferedReader` is a buffered character reader.

It's especially useful for reading text **line by line**.

</> Java



```
try (BufferedReader reader =
    new BufferedReader(new FileReader("data.txt"))) {

    String line;

    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
}
```

This is very useful for large text files because you don't need to load the entire file into memory.

Common interview question

BufferedReader vs Scanner?

For large text input, `BufferedReader` is generally preferred when you simply need efficient reading of characters/lines. `Scanner` provides convenient token parsing but has more parsing overhead.

5. File vs Path

This is a very common Java 8+ interview question.

File

`java.io.File` is the older API.

</> Java



```
File file = new File("data/test.txt");

if (file.exists()) {
    System.out.println(file.length());
}
```

Path

`java.nio.file.Path` is the newer NIO API.

</> Java



```
Path path = Path.of("data/test.txt");

if (Files.exists(path)) {
    System.out.println(Files.size(path));
}
```

Why Path is preferred?

NIO provides richer APIs through:

Path
Files
FileSystem



For example:

`</>` Java



```
Path source = Path.of("source.txt");
Path target = Path.of("target.txt");

Files.copy(source, target);
```

Interview answer

`File` is the legacy file-system abstraction from `java.io`, while `Path` is the newer NIO abstraction that works with the richer `Files` and `FileSystem` APIs.

6. java.io vs java.nio

java.io

Traditional I/O APIs:

```
InputStream
OutputStream
Reader
Writer
File
```



Primarily stream-oriented and traditionally blocking.

java.nio

Newer I/O APIs:

```
Path
Files
Channel
Buffer
Selector
```



Provides:

- Better file-system APIs
- Channels

- Buffers
- Non-blocking I/O
- Selectors
- More scalable networking models

7. What is NIO?

NIO = **New I/O**.

It was introduced to provide more flexible and scalable I/O capabilities.

The important concepts are:

NIO

- └ Channel
- └ Buffer
- └ Selector
- └ Files / Path



The traditional model is heavily stream-oriented:

InputStream → application



NIO commonly uses:

Channel ↔ Buffer ↔ Application



8. What is a Channel?

A `Channel` represents an open connection to an entity capable of I/O.

Examples:

</> **Java**

- `FileChannel`
- `SocketChannel`
- `ServerSocketChannel`
- `DatagramChannel`



Think of a channel as a **communication path**.

For example:

```
File
  ↑ ↓
FileChannel
  ↑ ↓
ByteBuffer
  ↑ ↓
Application
```



Example:

</> **Java**



```
try (FileChannel channel =
    FileChannel.open(Path.of("data.txt"))) {

    ByteBuffer buffer = ByteBuffer.allocate(1024);

    while (channel.read(buffer) != -1) {
        buffer.flip();

        // process buffer

        buffer.clear();
    }
}
```

9. What is a Buffer?

A **Buffer** is a container used to hold data while performing I/O.

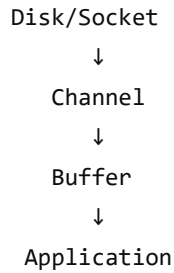
Common buffer:

</> **Java**



ByteBuffer

Think:



```
graph TD; A[Disk/Socket] --> B[Channel]; B --> C[Buffer]; C --> D[Application];
```

Disk/Socket
↓
Channel
↓
Buffer
↓
Application



A buffer has important properties:

capacity
position
limit



Example

</> Java

```
ByteBuffer buffer = ByteBuffer.allocate(1024);
```



This creates a buffer capable of holding 1024 bytes.

`flip()`

After writing data into the buffer:

</> Java

```
buffer.flip();
```



This prepares it for reading.

`clear()`

After consuming the data:

</> Java

```
buffer.clear();
```



This prepares the buffer to be filled again.

Easy interview explanation

A buffer is an in-memory area used to temporarily hold data during I/O operations, while a channel represents the connection through which the data moves.

10. Blocking vs Non-Blocking I/O

This is important for backend interviews.

Blocking I/O

Suppose your application reads from a socket:

`</> Java`

```
data = input.read();
```



If no data is available, the thread can wait.

Thread



read()



WAIT...



data arrives



continue



That thread is blocked while waiting.

Traditional Java I/O commonly uses this model.

11. Non-Blocking I/O

With non-blocking I/O, the thread doesn't necessarily wait for each operation to complete.

NIO provides mechanisms such as:

SocketChannel
Selector



Conceptually:



A selector can monitor multiple channels and notify the application when channels are ready for I/O.

This can be useful for servers handling many concurrent connections.

12. Blocking vs Non-Blocking

Blocking	Non-blocking
Thread may wait	Thread can continue
Simpler programming model	More complex
One thread can remain blocked	One thread can manage many channels
Traditional I/O commonly uses it	NIO supports it
Can require many threads for many waiting connections	Can be more scalable for many connections

Important interview nuance

Don't say:

"NIO is always non-blocking."

That's incorrect.

NIO supports **both blocking and non-blocking modes**, depending on the API/configuration.

For example, a `SocketChannel` can be configured as non-blocking.

13. How would you process a 10 GB file without loading it completely into memory?

🔥 Very important interview question.

Never do:

</> Java

```
String content = Files.readString(path);
```



for a 10 GB file.

You don't want:

10 GB File

↓

10 GB+ memory



Instead, process it incrementally.

Option 1 — BufferedReader

For a large text file:

</> Java

```
Path path = Path.of("large-file.txt");

try (BufferedReader reader = Files.newBufferedReader(path)) {

    String line;

    while ((line = reader.readLine()) != null) {
        process(line);
    }
}
```



Memory usage is approximately:

File = 10 GB



Memory:

buffer + current line + application state

Not:

10 GB file → 10 GB RAM



This is usually the simplest answer for a large text file.

14. Using InputStream for Binary Files

If it's binary data:

</> Java



```
try (InputStream input =
    new BufferedInputStream(
        Files.newInputStream(path))) {

    byte[] buffer = new byte[8192];

    int bytesRead;

    while ((bytesRead = input.read(buffer)) != -1) {
        process(buffer, bytesRead);
    }
}
```

The important part is:

</> Java



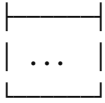
```
byte[] buffer = new byte[8192];
```

You're processing chunks instead of the entire file.

10 GB file



```
┌───┐
│ 8 KB │ → process
└───┘
┌───┐
│ 8 KB │ → process
└───┘
┌───┐
│ 8 KB │ → process
└───┘
```



15. Using FileChannel

For more control, NIO provides `FileChannel` .

</> **Java**



```
try (FileChannel channel =
    FileChannel.open(path)) {

    ByteBuffer buffer = ByteBuffer.allocate(64 * 1024);

    while (channel.read(buffer) != -1) {

        buffer.flip();

        while (buffer.hasRemaining()) {
            // process byte
            byte b = buffer.get();
        }

        buffer.clear();
    }
}
```

Here:

```
File
↓
FileChannel
↓
64 KB ByteBuffer
↓
Process
↓
reuse buffer
```



The 10 GB file never needs to exist completely in memory.

16. What if the file contains millions of records?

This is where a **5-year candidate** should go one level deeper.

Suppose:

10 GB CSV



You shouldn't do:

</> **Java**



```
List<String> allLines = Files.readAllLines(path);
```

because you're building a huge in-memory collection.

Instead:

```
Read line
  ↓
Parse line
  ↓
Validate
  ↓
Process
  ↓
Discard
  ↓
Read next line
```



For database processing:

```
10 GB CSV
  ↓
Read 1000 records
  ↓
Process batch
  ↓
Commit
  ↓
Read next 1000
```



This is also the general idea behind **streaming/batch processing**.

17. What about `Files.lines()` ?

For large text files, you can also use:

 `</> Java`

```
try (Stream<String> lines = Files.lines(path)) {  
    lines.forEach(this::process);  
}
```

This provides lazy line processing rather than creating a list containing every line.

But remember to **close the stream**, preferably with try-with-resources, because it is backed by an open file.

★ Interview Cheat Sheet

Concept	One-line answer
InputStream	Reads bytes
Reader	Reads characters
OutputStream	Writes bytes
Writer	Writes characters
BufferedInputStream	Buffers byte input to reduce underlying I/O calls
BufferedReader	Buffers character input and conveniently reads lines
File	Legacy file-system API
Path	Modern NIO file-system abstraction
NIO	Modern Java I/O APIs
Channel	I/O communication path
Buffer	Memory area used during I/O
Blocking I/O	Thread may wait for I/O

Concept**One-line answer**

Non-blocking I/O

I/O operation doesn't require the thread to wait in the same way

10 GB file

Process incrementally/chunk-by-chunk

★ **Best 5-year answer for the 10 GB question**

"I would never load the entire 10 GB file into memory. If it's a text file, I'd use `BufferedReader` or `Files.lines()` and process it line by line. If it's binary data, I'd use a buffered `InputStream` or `FileChannel` with a fixed-size `ByteBuffer`. If the processing involves a database, I'd additionally process records in bounded batches so that neither the file nor all records accumulate in memory."



What is JDBC?

JDBC driver types.

Statement vs PreparedStatement.

Why is PreparedStatement preferred?

SQL injection.

Connection pooling.

What happens if you don't close a DB connection?

What is transaction management?

Commit vs rollback.

Auto-commit.

Batch processing using JDBC.

[Show more](#) ✓

These are **very important JDBC + database interview questions for a 5-year Java/Spring Boot developer**. At your level, don't stop at definitions—connect JDBC concepts to performance, transactions, connection pools, and production failures.

1. What is JDBC?

JDBC = Java Database Connectivity.

It is a standard Java API that allows a Java application to communicate with relational databases.

Typical flow:

Java Application



JDBC



JDBC Driver



Database



For example:

Spring Boot



JDBC / JPA



PostgreSQL JDBC Driver



PostgreSQL



JDBC provides APIs such as:

</> Java

Connection

PreparedStatement

Statement

ResultSet



Example:

</> Java

```
try (Connection connection =
    dataSource.getConnection();

    PreparedStatement ps =
        connection.prepareStatement(
            "SELECT * FROM employee WHERE id = ?")) {

    ps.setLong(1, 100L);
```



```
try (ResultSet rs = ps.executeQuery()) {  
    while (rs.next()) {  
        System.out.println(rs.getString("name"));  
    }  
}  
}
```

2. JDBC Driver Types

Historically, JDBC drivers were categorized into **four types**.

Type	Name	Description
Type 1	JDBC-ODBC Bridge	JDBC → ODBC → DB
Type 2	Native API	JDBC → native DB library
Type 3	Network Protocol	JDBC → middleware → DB
Type 4	Thin Driver	JDBC → database protocol

Type 4 is what modern Java applications generally use.

For example:

Java Application



PostgreSQL JDBC Driver



PostgreSQL



The driver communicates directly using the database's protocol.

Interview answer

"JDBC drivers were historically classified into four types. Type 4, the pure Java driver that directly communicates with the database protocol, is the commonly used approach in modern applications."

3. Statement vs PreparedStatement

Statement

You construct the SQL string yourself.

</> Java

```
Statement stmt = connection.createStatement();

ResultSet rs =
    stmt.executeQuery(
        "SELECT * FROM employee WHERE id = 10");
```

PreparedStatement

SQL contains parameters:

</> Java

```
PreparedStatement ps =
    connection.prepareStatement(
        "SELECT * FROM employee WHERE id = ?");

ps.setInt(1, 10);

ResultSet rs = ps.executeQuery();
```

Comparison:

Statement	PreparedStatement
SQL constructed dynamically	SQL template + parameters
More vulnerable to SQL injection	Helps prevent SQL injection
Less suitable for repeated parameterized queries	Better for repeated execution
Values concatenated into SQL	Values bound separately

4. Why is PreparedStatement preferred?

There are **three** major reasons.

1. SQL injection protection

Instead of:

```
</> Java   
  
String sql =  
    "SELECT * FROM users WHERE name = '" + name + "'";
```

use:

```
</> Java   
  
PreparedStatement ps =  
    connection.prepareStatement(  
        "SELECT * FROM users WHERE name = ?");  
  
ps.setString(1, name);
```

The value is treated as a parameter rather than being directly interpreted as SQL syntax.

2. Better handling of parameters

You can use:

```
</> Java   
  
ps.setString(...)  
ps.setInt(...)  
ps.setLong(...)  
ps.setDate(...)
```

instead of manually building SQL strings.

3. Repeated execution

If you're executing the same SQL structure with different values:

```
  
INSERT employee 1  
INSERT employee 2  
INSERT employee 3  
...
```

a `PreparedStatement` is a much better fit.

Some databases/drivers can also reuse prepared statement execution plans, though the exact behavior depends on the driver/database configuration.

5. What is SQL Injection?

SQL injection happens when untrusted input changes the intended SQL command.

Bad approach:

```
</> Java   
  
String sql =  
    "SELECT * FROM users WHERE username = '"  
    + username + "'";
```

The user input becomes part of the SQL syntax.

Safe approach

```
</> Java   
  
PreparedStatement ps =  
    connection.prepareStatement(  
        "SELECT * FROM users WHERE username = ?");  
  
ps.setString(1, username);
```

The parameter is bound separately.

Interview answer

"SQL injection is an attack where untrusted input is interpreted as part of a SQL statement. PreparedStatement prevents the common form of this problem by separating SQL structure from parameter values."

6. What is Connection Pooling?

Creating a database connection is relatively expensive.

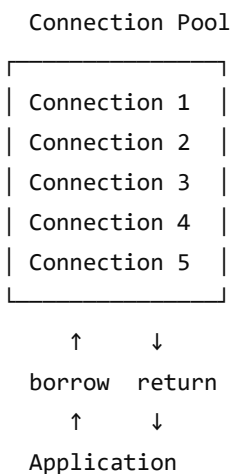
Without pooling:

Request 1
 ↓
 Create DB connection
 ↓
 Execute query
 ↓
 Close connection

Request 2
 ↓
 Create DB connection
 ↓
 Execute query
 ↓
 Close connection



With connection pooling:



The application **borrow**s a connection, uses it, and returns it to the pool.

In Spring Boot applications, **HikariCP** is commonly used as the connection pool.

7. What happens if you don't close a DB connection?

This is a very important production question.

Suppose your pool has:

Maximum connections = 10



And your application does:

</> **Java**

```
Connection connection =  
    dataSource.getConnection();  
  
// query  
  
// forgot to close()
```

That connection may remain unavailable to the pool.

Repeat this across requests:

```
Request 1 → connection leaked  
Request 2 → connection leaked  
Request 3 → connection leaked  
...  
Request 10 → connection leaked
```



Eventually:

```
Pool  
├─ Connection 1 ✗ leaked  
├─ Connection 2 ✗ leaked  
├─ Connection 3 ✗ leaked  
├─ ...  
└─ Connection 10 ✗ leaked
```



Now new requests can't obtain connections.

This is called a **connection leak**.

Correct approach

Use try-with-resources:

</> **Java**

```
try (Connection connection =  
    dataSource.getConnection()) {  
  
    // database work  
}
```

With a pool, `close()` normally means **return the connection to the pool**, not necessarily physically close the underlying database connection.

That's an excellent 5-year interview point.

8. What is Transaction Management?

A transaction groups multiple database operations into a logical unit.

Suppose you're transferring money:

```
Account A
  ↓
Debit ₹100
  ↓
Account B
  ↓
Credit ₹100
```



Both operations should succeed together.

```
Transaction
├─ Debit A
└─ Credit B
```



If both succeed:

```
COMMIT
```



If something fails:

```
ROLLBACK
```



The goal is to maintain the database's consistency.

In Spring:

```
</> Java

@Transactional
public void transferMoney() {
```



```
debitAccount();  
  
creditAccount();  
}
```

9. Commit vs Rollback

Commit

Makes transaction changes permanent.

```
BEGIN  
↓  
UPDATE  
↓  
INSERT  
↓  
COMMIT
```



Changes become committed according to the database's transaction semantics.

Rollback

Undoes the uncommitted changes in the current transaction.

```
BEGIN  
↓  
UPDATE  
↓  
INSERT  
↓  
ERROR  
↓  
ROLLBACK
```



10. What is Auto-Commit?

JDBC connections commonly start with auto-commit enabled unless configured otherwise.

When auto-commit is enabled:

</> Java



```
connection.setAutoCommit(true);
```

each individual statement generally forms its own transaction.

Conceptually:

UPDATE

↓

COMMIT

INSERT

↓

COMMIT

DELETE

↓

COMMIT



If you need multiple operations to form **one transaction**:

</> Java



```
connection.setAutoCommit(false);
```

```
try {
```

```
    // operation 1
```

```
    // operation 2
```

```
    connection.commit();
```

```
} catch (Exception e) {
```

```
    connection.rollback();
```

```
}
```

Conceptually:

```
AutoCommit = false
```



```
BEGIN
```

↓

```
Operation 1
```

↓

```
Operation 2
```

```
↓  
Operation 3  
↓  
COMMIT
```

or:

```
Operation 1  
↓  
Operation 2  
↓  
ERROR  
↓  
ROLLBACK
```



11. Batch Processing Using JDBC

Suppose you need to insert 100,000 records.

Bad approach:

```
INSERT 1  
INSERT 2  
INSERT 3  
...  
INSERT 100000
```



with separate execution/round trips.

Instead, use JDBC batching.

```
</> Java  
  
String sql =  
    "INSERT INTO employee(id, name) VALUES (?, ?)";  
  
try (PreparedStatement ps =  
    connection.prepareStatement(sql)) {  
  
    for (Employee employee : employees) {  
  
        ps.setLong(1, employee.getId());  
        ps.setString(2, employee.getName());  

```



```
        ps.addBatch();  
    }  
  
    ps.executeBatch();  
}
```

Why batching helps

It can reduce the overhead associated with executing many individual statements and database round trips.

For very large workloads, you should also consider **batch size**, transaction boundaries, memory usage, and database-specific limits.

12. How does Connection Pooling Improve Performance?

Without pooling:

```
Request  
↓  
TCP connection to DB  
↓  
Authentication / setup  
↓  
Database connection  
↓  
Query  
↓  
Close
```



This connection establishment has overhead.

With pooling:

```
Pool  
↓  
Borrow existing connection  
↓  
Query  
↓  
Return to pool
```



Therefore, pooling:

- Reduces connection creation overhead
- Reduces connection establishment latency
- Allows controlled concurrency
- Reuses established DB connections
- Helps protect the database from excessive connection creation

Important

Connection pooling does **not** make an individual SQL query inherently faster.

It reduces the overhead around acquiring database connections and controls concurrent DB access.

13. What happens when DB Connection Pool is Exhausted?

🔥 Very important production question.

Suppose:

```
maximumPoolSize = 10
```



and all 10 connections are currently being used:

```
Connection 1 → Request A  
Connection 2 → Request B  
Connection 3 → Request C  
...  
Connection 10 → Request J
```



Now:

```
Request K  
↓  
getConnection()  
↓  
No connection available  
↓  
Wait
```



The pool may wait up to its configured connection acquisition timeout.

If a connection becomes available:

Connection returned



Request K gets it



If the timeout expires:

Connection acquisition timeout



and the application receives an exception.

With HikariCP, this is typically a `SQLTransientConnectionException` when a connection cannot be obtained within the configured timeout.

14. Why can a connection pool become exhausted?

This is where interviewers expect production thinking.

Cause 1 — Connection leak

`getConnection()`



forgot `close()`



Connections aren't returned.

Cause 2 — Slow queries

Request



DB connection



Very slow query



connection occupied for 30 seconds



Many requests can exhaust the pool.

Cause 3 — Long transactions



```
BEGIN
↓
query
↓
business processing
↓
external API call
↓
more processing
↓
COMMIT
```

Holding a DB connection while doing unrelated work is dangerous.

Cause 4 — Too many concurrent requests

For example:



```
1000 requests
↓
Only 20 DB connections
↓
Many requests waiting
```

Cause 5 — Database itself is overloaded

Even if your application is healthy:



```
Application
↓
Connection Pool
↓
Database
↓
CPU / locks / I/O overloaded
```

Queries become slow, connections stay occupied longer, and the pool eventually gets exhausted.

★ Very Important Production Chain

A great 5-year candidate should be able to explain this:

High Traffic
↓
Many requests
↓
Connection Pool
↓
All connections occupied
↓
Requests wait
↓
Pool acquisition timeout
↓
Application errors



But the **root cause** might be:

Slow SQL
OR
Connection leak
OR
Long transaction
OR
DB overloaded
OR
Too much concurrency



So don't simply "increase the connection pool."

Increasing:

10 → 100 → 500



can actually make the database situation worse if the DB can't handle that concurrency.

★ Interview Cheat Sheet

Question	Strong short answer
JDBC	Standard Java API for communicating with relational databases
JDBC driver	Database-specific implementation that allows JDBC to communicate with the DB
Statement	Executes SQL constructed as a string
PreparedStatement	Parameterized SQL; safer and better for repeated parameterized execution
SQL Injection	Untrusted input alters SQL semantics
Connection Pool	Reuses a controlled set of DB connections
Don't close connection	Can cause connection leaks and pool exhaustion
Transaction	Logical unit of DB operations
Commit	Permanently commits transaction changes
Rollback	Discards uncommitted transaction changes
Auto-commit	Each statement is generally committed independently
Batch processing	Groups multiple statements for more efficient execution
Pool performance	Avoids repeated connection establishment and controls concurrency
Pool exhausted	Requests wait for a connection and may eventually time out

★ One answer worth memorizing

If asked:

"Your production API suddenly starts throwing database connection timeout errors. What would you check?"

A strong answer:

"First I'd check connection-pool metrics—active, idle, and pending connections—and the pool acquisition timeout. Then I'd investigate connection leaks, slow queries, long-running transactions, database locks, and database CPU/I/O saturation. I'd also check whether traffic or application concurrency has increased. I wouldn't immediately increase the pool size because that could simply move the bottleneck to the database. I'd identify why connections are staying occupied for too long and fix the underlying issue."



ChatGPT can make mistakes. Check important info.